Time to Detected Contradiction in Saved AI Code Explanations

Anonymous Author(s)

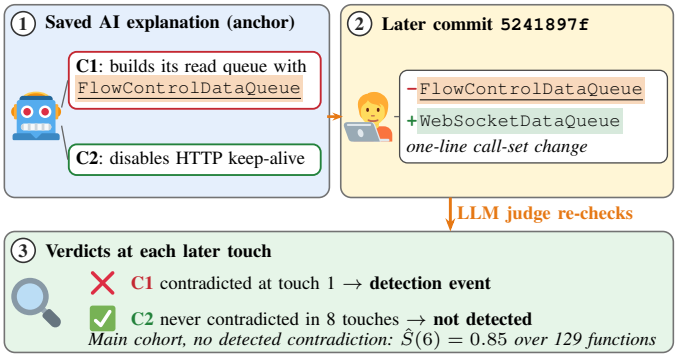


Fig. 1. Hand-picked aiohttp case, `_post_start`, touch 1 = `5241897f` (shortened claims). The judge contradicts C1; C2 remains uncontradicted. Icons: Twemoji (CC-BY 4.0).

***Abstract*—Saved AI code explanations can become stale as code changes. We present an observational, claim-level instrument whose endpoint is first LLM-judge-detected contradiction, not semantic failure. Among anchor-supported claims in 129 functions from 11 Python projects, freedom from detection is $\hat{S}(6) = 0.85$ [0.81,0.87] after six touches under Qwen; a full Llama replay gives $\hat{S}_{\text{Llama}}(6) = 0.613$ [0.573,0.651]. An independent rater, self-reported as blinded, relabeled all 218 observed Qwen contradiction rows through touch six, giving $\hat{S}(6) = 0.923$–0.925under two unresolved-label rules. This one-sided single-rater sensitivity removes observed false positives but cannot recover missed contradictions or establish semantic truth. Two independently blinded human raters with self-reported provenance agree on 99.50% of 400 judgments ($\kappa = 0.992$), with two disagreements adjudicated independently.**

***Index Terms*—code explanation, program comprehension, documentation decay, survival analysis, LLM**

I. Introduction

LLM-generated explanations of code are increasingly kept rather than discarded: tools such as RepoAgent [1], DocAgent [2] and RepoDoc [3] write them into repository documentation, and developers paste them into reviews and wikis. The code keeps changing; the saved text does not. Whether such text is still true later is a program-comprehension question, because a fluent but stale statement seeds a wrong mental model [4]. Yet explanations are evaluated almost only when they are produced [5]–[7], and human-comment research treats a comment as one unit that is consistent or not [8], [9].

We treat a saved explanation as a set of *claims* with observed detection times. Each claim ("`draw` raises `InvalidArgument` inside `find`") is checked against later versions of its function. The endpoint is its first judge-detected contradiction; $S(k)$ denotes freedom from this detection after k local code-changing commits (*touches*). It is not a semantic-survival estimate. This pilot asks whether the instrument can be audited and what descriptive detection curve it produces under its stated observation rules.

The study began with qualitative expectations about churn, claim type, whole-text checks and docstrings, but both the operational tests and the implementation plan changed after data collection began (Section IV). We therefore report every covariate and comparator result as exploratory diagnostics, rather than as confirmatory hypothesis tests.

What is the new idea? Follow first judge-detected contradictions of retained AI claims over real histories, separating anchor verdicts, later detections and censoring.
Why is it new? Prior work evaluates explanations once [5], [6], studies decay of human comments, links or references [8], [10], [11], or builds refresh systems [3], [12]. To our knowledge, no study estimates survival of individual claims of retained AI explanations.
Single most related paper: Khatib et al. [5] test whether a *newly generated* summary reflects synthetic mutations; we keep the *original* explanation and follow real commits.

Contributions. (1) An operational detection-time measure with censoring and explicit observation assumptions. (2) Cached longitudinal records, model and single-rater human-label sensitivity analyses, and a two-human-rater audit with blind adjudication. (3) Exploratory diagnostics and a validation agenda. Code and data: https://claim-survival-artifact.dimpledshrimp2.chatgpt.site.

II. Background and Related Work

Decay of human documentation. Comments and code often fail to co-evolve [13]; stale comments mislead developers [14]; Wen et al. catalogued the code-change types that introduce comment inconsistencies [8]. Radmanesh et al. follow inconsistencies over time and find their bug risk highest right after they appear [15]. Comment links decay [10], outdated code-element references are widespread [11], and documentation churn trails code churn [16]. Survival analysis is established in software engineering, e.g. for self-admitted technical debt [17] and for AI-generated code [18]. We study the decay of *retained AI explanations* at the level of individual claims, which these works do not.

Just-in-time (JIT) detection. Detectors decide per commit whether a comment became inconsistent [9], [19]. CARL-CCI uses structured diffs [20]; CCISolver uses LLMs [21]. Our exploratory whole-text prompt diagnostic is not a test of these

published systems; a frozen-input comparison remains future work.

Faithfulness of generated summaries. Khatib et al. inject behaviour-changing mutations and test whether *newly generated* LLM summaries reflect them [5]. ETF [7] and ReFEree [6] decompose summaries into checkable units, as does atomic-fact scoring [22]; Sistla et al. check formalised code-reasoning claims with program analysis [23]. We apply claim decomposition *longitudinally* to an explanation that is kept, over its function's history.

Keeping documentation current. RepoAgent [1], DocAgent [2], RepoDoc [3] and CODENS [12] update repository documentation; CoDocBench pairs code/docstring changes [24]. READU checks README inconsistencies against internal/external facts [25]. Such grounding and ReFEree's dependency-aware segments [6] could strengthen our local judge; neither is evaluated here. Our target is detection time for retained claims.

III. Approach: Claim-Survival Analysis

Construct and decomposition. An explanation E of function f is written at anchor c_0. The prompt requires each claim to be a self-contained, source-checkable fact naming f, with no added or merged facts, original wording where possible, and a 12-claim cap. Each receives one type: *structural* (signature, parameters, calls), *data-flow*, *behavioural* (conditions, exceptions, side effects), or *intent*. These instructions do not guarantee atomicity or coverage. A *touch* v_k is a later mainline commit changing f's code tokens (docstrings, comments and layout removed). Git file renames are followed; fallback matching requires a unique new same-name candidate in the same file with text similarity ≥ 0.7. Unmatched tracks are censored; matching accuracy is unvalidated. A judge $J(c, f@v_k) \in \{\text{SUP}, \text{CON}, \text{UNR}\}$ uses docstring-free code, the diff and tree-sitter [26] facts (parameters, calls, returns, raises).

Cohort, event, censoring. Only anchor-SUP claims enter; anchor contradictions are initial detected errors. The event is the *first* CON, so later support cannot undo it. Follow-up is censored at $K_{\max}$, the pinned head, or unmatched removal/rename. Kaplan–Meier [27] estimates $S(k) = \Pr[T > k]$ for this detection time, conditional on anchor support. We report $\hat{S}(K)$ and the time to 10% detections; the half-life is not reached. *Unresolved verdicts:* primary analysis keeps UNR claims at risk without an event; sensitivity scenarios instead censor immediately before the first UNR, or count it as an event. Neither scenario bounds semantic survival; unresolved is not supported. Reversible-state modeling targets a different estimand and needs comparable post-event observations absent from the alive-only archive.

Hazard model. Time is discrete with heavy ties. Our primary model is therefore a discrete-time complementary log-log regression, which is a grouped-time proportional-hazards model:

$$1 - h(k \mid x) = \exp\big(-\exp(\alpha_k + \beta^\top x_k)\big), \qquad (1)$$

with touch-index effects α_k. Covariates are claim type, $\log(1+\text{lines changed})$ in the function and in its file, and heuristic change-type flags obtained by diffing parser facts. Every interval and Wald test uses the function-clustered sandwich covariance. As robustness checks we fit an unpenalised Cox model with time-varying covariates in counting-process form [28]–[30], and test proportional hazards with covariate$\times \log k$ interactions.

Placebo sensitivity. Rejudging unchanged code gives a scenario with constant false-contradiction rate ϵ: $\max(0, (h_k - \epsilon)/(1 - \epsilon))$. Error adjustment would require placebo rates to transfer to real-diff prompts, independent repeated errors conditional on risk, and no missed contradictions. Contexts differ and judges disagree both ways; these assumptions are unvalidated. The curve is a sensitivity scenario, not a calibrated correction.

Exploratory comparators. A whole-explanation JIT check by the same model uses three prompt versions (binary v1 and v2, graded 0–10 v3); a non-LLM *identifier check* flags a claim once an identifier it names, present at c_0, is missing from the code. We also decompose and judge the function's anchor docstring in the same shuffled list as the LLM claims. These are implementation diagnostics, not published-baseline comparisons or causal authorship tests.

IV. Preliminary Evaluation

Subjects and count ledger. We cloned 14 active Python projects with full history, pinned each head commit, and fixed the anchor as the last mainline commit before 2024-11-01. *Main cohort:* non-test functions of 6–120 lines with $\geq$6 touches up to the head (151 candidates; seeded cap of 20 per project), giving 129 functions in 11 projects, followed for up to $K_{\max}$=12 touches: 1038 raw candidate function–commit touches in 671 distinct commits, median head distance 738 days (Table I). Removing 9 quote/comma-only and 8 parenthesis-only re-wraps leaves 1020 effective touches; the churn diagnostic then has 1014 main-LLM at-risk function–touch rows after its analysis restrictions. These are different denominators, not conflicting counts. One later-added `@overload` stub ends a follow-up; duplicate names and overload stubs are excluded. As the main cohort selects on future activity, a seeded random cohort of 80 eligible anchor functions was drawn; excluding vendored code (2023 of 12926 functions; one main-cohort function too) leaves 61 non-vendored functions and 21 ever-touched functions. A separate clean redraw is reported only as supplemental evidence.

Models. One model, qwen2.5-coder:14b [31] (temperature 0, cached), generates explanations (from code *with* docstring, prompted to cover parameters, helpers and errors), decomposes, judges and runs the JIT checks; 72 of 129 explanations hit the 12-claim cap, and the judge prompt ("written about an earlier version") may prime. As a *reliability proxy, not human annotation*, claude-haiku-5-5 re-judged identical claim lists (96 of 125 pairs still valid, a stability-selected subset).

TABLE I

SUBJECTS. MAIN COHORT: FUNCTIONS, FUNCTION–COMMIT TOUCHES (UP TO 12 EACH) AND DISTINCT COMMITS; RAND.: RANDOM-COHORT FUNCTIONS (OTHERS: RICH, FLASK, HTTPX).

Project	Func.	Touches	Commits	w/ doc	LLM cl.	Doc cl.	Rand.
pytest	9	66	51	4	94	30	5
pydantic	20	171	96	18	223	125	2
black	15	123	88	10	165	36	2
click	5	41	28	4	56	18	3
attrs	1	6	6	1	9	5	0
pip	16	125	69	4	181	16	7
hypothesis	19	172	95	5	196	25	6
sphinx	12	96	64	3	131	7	11
pylint	3	24	23	3	32	4	8
aiohttp	20	150	103	9	227	21	0
textual	9	64	48	4	101	23	12
others	–	–	–	–	–	–	5
Total	129	1038	671	65	1415	310	61

TABLE II

FREEDOM FROM JUDGE-DETECTED CONTRADICTION AMONG ANCHOR-SUPPORTED CLAIMS. MAIN COHORT SELECTS FUTURE-ACTIVE, TRACKABLE FUNCTIONS; PLACEBO ADJUSTMENT IS A SCENARIO. HRS VS. DATA-FLOW.

Claims	n	Events$_{\le 12}$	$\hat{S}(6)$ [95% CI]	$t_{0.9}$	HR [95% CI]
Main (all)	1354	250	0.85 [0.81, 0.87]	4	–
placebo scenario			0.86 [0.83, 0.89]		–
Random (k=1)	219	12	n.r.	–	–
Structural	443	93	0.81 [0.77, 0.85]	3	1.06 [0.81, 1.38]
Data-flow	489	102	0.84 [0.80, 0.87]	5	1 (ref.)
Behavioural	338	49	0.88 [0.84, 0.91]	5	0.73 [0.51, 1.05]
Intent	84	6	0.93 [0.87, 0.99]	n.r.	0.32 [0.13, 0.79]

Protocol changes. The anchor date, projects, touch threshold, project cap, judging format/context, JIT prompts, analysis models, random cohort, placebos, and exclusions changed after the study began. The complete dated change log and frozen inputs are in the artifact. Consequently, we do not treat p-values, Holm adjustment, or any stated decision rule as confirmatory evidence. The archived ledger contains 3818 local and 205 hosted calls; the separate reviewer-follow-up replay and mutation calls are reported in the artifact experiment ledgers.

Initial judge-detected errors. At the anchor the judge contradicted 1.3% of LLM claims and 13.9% of docstring claims. An older assistant-only audit lacks its claim-level labels and is not used for any result here. The separately released human-audit records, matrix and sampling weights are available in the artifact.

Detection-time survival. Table II and Fig. 2 give Kaplan–Meier estimates. In the future-active, trackable main cohort, 250 of 1354 anchor-supported LLM claims had a detected contradiction, $\hat{S}(6) = 0.85$ [0.81, 0.87], and 10% had a detected contradiction after $t_{0.9} = 4$ touches. The half-life is not reached by touch 12 ($\hat{S} = 0.76$); constant-hazard extrapolations are retained only in the artifact. Only 219 random-cohort claims (21 functions) are ever at risk (hazard 2.8%), and from k=3 only 24 in 2 functions; by our rule ($\ge$30 claims in $\ge$5 functions) its curve is reported only to

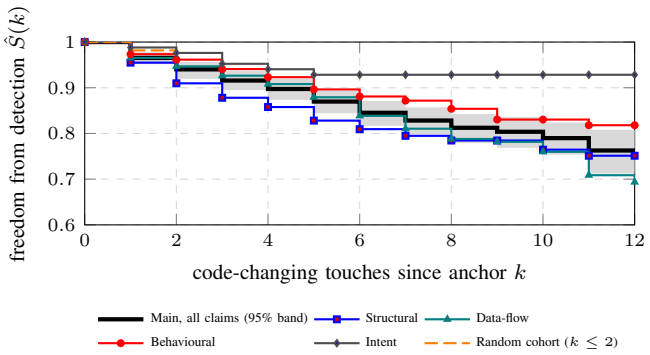


Fig. 2. Kaplan–Meier freedom from judge-detected contradiction: future-active main cohort (all, cluster-bootstrap band, by type) and random cohort.

k=2 ($\hat{S} = 0.97$). *Calendar time:* with removal censored, $\hat{S}(365\,\mathrm{d})$ is 0.98 (random) vs. 0.89 (main); counting removal or rename as death of all claims, it is 0.92 [0.84, 0.98] vs. 0.88. Competing risks at 365 days: claim death 1.5% vs. 11.2%, removal 6.7% vs. 0.8%. The random-cohort contrast rests on 4 removed vs. 4 contradicted functions (difference 5.2 pp [-0.5, 12.3]), removed files include release scripts, per-touch hazards are similar, and eligibility builds in the main cohort's low removal rate: the comparison does not establish an ordering in the population. *Placebo sensitivity:* 5 of 1780 placebo looks were contradicted (0.3%), all in the anchor retest whose prompt says "no code change"; real code-unchanged versions gave 0 of 426 (docstring arm retest: 3 of 233). Under the stated error model, $\hat{S}(6) = 0.86$ [0.83, 0.89] (joint bootstrap). The implied 9.3% attribution is assumption-dependent, not a measured false-death fraction. Among 53 main claims with unresolved follow-up, censoring before the first unresolved verdict gives $\hat{S}(6) = 0.849$; counting it as an event gives 0.817 (primary 0.85). These exploratory scenarios are not semantic-survival bounds; censoring may be informative.

Exploratory diagnostics. Churn has $\rho = 0.25$ [0.19, 0.31] over 1014 analysis rows (82.5% without an event), AUC 0.68, and HR 1.56 [1.19, 2.03] per inter-quartile range of function lines. Claim type is similarly unstable: the apparent intent contrast (HR 0.32 [0.13, 0.79] vs. data-flow) rests on 6 events, 4 in three click pager functions; excluding them leaves 2 events. We therefore make no claim-type ordering. Whole-text prompts v1/v2 flag every queried touch (including unchanged anchors) and v3 barely separates outcomes (AUC 0.58 [0.55, 0.61]); this is a failed implementation check, not a comparison with published JIT systems. The docstring contrast is also descriptive only: generator exposure, content/granularity and left-truncation confound it. The artifact retains all coefficients, resampling outputs and negative results.

Reliability and human audit. Reweighted to the population strata (155/619 touches with/without a death), the archived judges agree on 65.9% of verdicts ($\kappa = 0.09$; $\kappa = 0.21$ for contradicted vs. not). claude-haiku-5-5 confirms 51.9% of bulk contradictions and contradicts 6.7% of claims the bulk judge kept: this proxy disagrees in both directions and cannot

TABLE III
TWO-RATER AUDIT REFERENCE OF 400 RECORDED JUDGMENTS. ROWS ARE BULK-MODEL LABELS; COLUMNS ARE FINAL AUDIT LABELS.

	SUP	CON	UNR
Bulk SUP	132	1	0
Bulk CON	0	134	0
Bulk UNR	28	2	103

calibrate $\hat{S}$. Separately, two self-reported blinded raters audited the same 400 records stratified by stage and bulk verdict. They agree on 99.50% ($\kappa = 0.992$; weighted agreement 99.980%), with 2 disagreements resolved by a self-reported independent blinded adjudicator (Table III). The resulting reference labels all 134 sampled bulk-CON records as CON, yielding observed agreement 134/134 (exact 95% lower limit 97.3%) in that sampled stratum; among 297 resolved reference labels, bulk-CON sensitivity is 134/137=97.8%. These are not semantic PPV/recall: post-plan sampling, unresolved labels, self-reported blinding, and possible rater bias prevent that interpretation.

V. DISCUSSION AND RESEARCH AGENDA

What the curves record. By touch six, 208 primary-Qwen detections occur in 150 function–touch groups; 97 occur in 39 groups with at least two detected claims. Claim-level intervals are therefore clustered. A full Llama replay covers 129 functions and 1290 anchor-supported claims: $\hat{S}_{\text{Llama}}(6) = 0.613$ [0.573,0.651]. Qwen–Llama anchor agreement is 87.19% ($\kappa = 0.241$; 1,725 labels). These are judge-specific detection curves, not semantic-survival estimates.

Targeted disagreement checks. One independent human rater, self-reported as blinded to model labels and outcomes and distinct from A1/A2, reviewed 170 deliberately selected disagreements. In complete anchor strata, 75/77 Qwen-SUP/Llama-CON rows were human-supported, while 9/37 Qwen-CON/Llama-SUP rows were human-contradicted; an expanded anchor-and-commit Qwen-CON/Llama-SUP set yielded 16/60 contradicted. These targeted frames differ from the 400-record audit and are neither population PPV/recall nor a curve correction.

Mutation checks. A synthetic suite detects 100.0% of 100 known changes [96.3–100% Wilson] with 0.0% false positives among 100 unchanged controls. On 25 subtler mutations in five families, the primary judge returned 22 CON, 2 SUP, and 1 UNR. Both results are targeted judge-sensitivity checks, not recall on repository histories.

Human labels. A1 and A2 independently labeled 400 blinded records and agreed on 99.50% ($\kappa = 0.992$; weighted agreement 99.980%); a third human adjudicator resolved 2 disagreements from a blind packet. The event-level rater overlaps A1/A2 on 55 Qwen-CON rows but labeled only 23 as contradicted, so A1/A2 agreement does not transfer to that rater/frame. Provenance, independence, and blinding statements are self-reported; no calibration session is documented.

Event-level human-label sensitivity. A new independent rater labeled all 218 unique observed Qwen-CON rows through touch six: 102 contradicted, 114 supported, and 2 unresolved. The resolved-row confirmation proportion is 47.2% [40.7,53.9% Wilson]. Substitution gives $\hat{S}(6) = 0.923$ when human UNR is an event and $\hat{S}(6) = 0.925$ when censored (function-cluster bootstrap 0.904–0.940), versus Qwen's 0.85. This removes observed Qwen false positives only; illustrative homogeneous recall .88/.70 gives 0.913/0.892. It is a one-sided, single-rater sensitivity analysis, not a semantic correction.

Limits and agenda. Future-conditioned selection, name-based matching, informative censoring, one generator/prompt, 11 Python projects, sparse random-cohort events, and non-local dependency changes limit generalization. Independent event-level replication, parser-free rejudging, published JIT comparators, refresh cost/coverage, and dependency tracking remain open; no developer-benefit or operational-cost advantage is established.

VI. CONCLUSION

Following retained claims yields a censoring-aware curve of first judge-detected contradictions in future-active, trackable functions. A full Llama replay and a single-rater, event-level human-label sensitivity analysis show that the Qwen curve is measurement-dependent; neither is a semantic-error estimate. Two-rater agreement with blind adjudication describes label consistency, not semantic recall. This pilot does not estimate semantic failure times, refresh benefits, or developer impact. The artifact records the inputs, code, labels and limitations needed to inspect these claims; its reviewer URL is an account-hosted anonymous release, not a DOI archive.

AI-USE DISCLOSURE

LLMs (qwen2.5-coder:14b and Llama 3.1 8B via Ollama, plus claude-haiku-5-5) are experimental subjects and judges. An AI coding assistant implemented the artifact, performed the identified automated audits, and drafted this text. Statistics are rendered from saved outputs; AI audits are not human evaluation.

References

[1] Q. Luo, Y. Ye, S. Liang, Z. Zhang, Y. Qin, Y. Lu, Y. Wu, X. Cong, Y. Lin, Y. Zhang, X. Che, Z. Liu, and M. Sun, "RepoAgent: An LLM-powered open-source framework for repository-level code documentation generation," in *Proc. EMNLP: System Demonstrations*, 2024.

[2] D. Yang, A. Simoulin, X. Qian, X. Liu, Y. Cao, Z. Teng, and G. Yang, "DocAgent: A multi-agent system for automated code documentation generation," in *Proc. ACL: System Demonstrations*, 2025.

[3] D. Xu, M. Liu, X. Wang, J. Zhong, and Z. Zheng, "RepoDoc: A knowledge graph-based framework to automatic documentation generation and incremental updates," arXiv:2604.26523, 2026.

[4] A. von Mayrhauser and A. M. Vans, "Program comprehension during software maintenance and evolution," *IEEE Computer*, vol. 28, no. 8, pp. 44–55, 1995.

[5] L. Khatib, M. Pu, B. Vasilescu, and M. Nagappan, "Using mutation-analysis to examine an LLM's ability to summarize code," arXiv:2602.17838, 2026.

[6] S. Bae, C. Na, J. Lee, Y. Lee, Y. Choi, and J.-H. Lee, "ReFEree: Reference-free and fine-grained method for evaluating factual consistency in real-world code summarization," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026, arXiv:2604.10520.

[7] K. Maharaj, V. Munigala, S. G. Tamilselvam, P. Kumar, S. Sen, P. Kodeswaran, A. Mishra, and P. Bhattacharyya, "ETF: An entity tracing framework for hallucination detection in code summaries," arXiv:2410.14748, 2024.

[8] F. Wen, C. Nagy, G. Bavota, and M. Lanza, "A large-scale empirical study on code-comment inconsistencies," in *Proc. IEEE/ACM Int. Conf. Program Comprehension (ICPC)*, 2019, pp. 53–64.

[9] S. Panthaplackel, J. J. Li, M. Gligoric, and R. J. Mooney, "Deep just-in-time inconsistency detection between comments and source code," in *Proc. AAAI Conf. Artificial Intelligence*, 2021.

[10] H. Hata, C. Treude, R. G. Kula, and T. Ishio, "9.6 million links in source code comments: Purpose, evolution, and decay," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2019, pp. 1211–1221.

[11] W. S. Tan, M. Wagner, and C. Treude, "Detecting outdated code element references in software repository documentation," *Empirical Software Engineering*, vol. 29, no. 1, p. 5, 2024.

[12] A. Kelious, C. Tahri, and E. Bardet, "CODENS: Transforming code changes into living, accessible, and queryable documentation," arXiv:2607.18356, 2026.

[13] B. Fluri, M. Würsch, and H. C. Gall, "Do code and comments co-evolve? On the relation between source code and comment changes," in *Proc. Working Conf. Reverse Engineering (WCRE)*, 2007, pp. 70–79.

[14] L. Tan, D. Yuan, G. Krishna, and Y. Zhou, "/*iComment: Bugs or bad comments?*/," in *Proc. ACM Symp. Operating Systems Principles (SOSP)*, 2007, pp. 145–158.

[15] S. Radmanesh, A. Imani, I. Ahmed, and M. Moshirpour, "Investigating the impact of code comment inconsistency on bug introducing," arXiv:2409.10781, 2024.

[16] A. Hovhannisyan, Y. Fan, G. Rodriguez-Perez, and R. G. Kula, "Linking code and documentation churn: Preliminary analysis," arXiv:2410.05992, 2024.

[17] B. A. Muse, C. Nagy, A. Cleve, F. Khomh, and G. Antoniol, "FIXME: Synchronize with database! An empirical study of data access self-admitted technical debt," *Empirical Software Engineering*, vol. 27, no. 6, p. 130, 2022.

[18] M. Rahman and E. Shihab, "Will it survive? Deciphering the fate of AI-generated code in open source," arXiv preprint arXiv:2601.16809, 2026.

[19] Z. Liu, X. Xia, D. Lo, M. Yan, and S. Li, "Just-in-time obsolete comment detection and update," *IEEE Trans. Software Engineering*, vol. 49, no. 1, pp. 1–23, 2023.

[20] P. Nguyen, A. M. T. Bui, and P. T. Nguyen, "Larger is not always better: Leveraging structured code diffs for comment inconsistency detection," arXiv:2512.19883; SANER 2026 short paper, 2026.

[21] R. Zhong, Y. Huo, W. Gu, J. Kuang, Z. Jiang, G. Yu, Y. Li, D. Lo, and M. R. Lyu, "CCISolver: End-to-end detection and repair of method-level code-comment inconsistency," arXiv:2506.20558, 2025.

[22] S. Min, K. Krishna, X. Lyu, M. Lewis, W. tau Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[23] M. Sistla, G. Balakrishnan, P. Rondon, J. Cambronero, M. Tufano, and S. Chandra, "Towards verified code reasoning by LLMs," arXiv:2509.26546, 2025.

[24] K. Pai, P. Devanbu, and T. Ahmed, "CoDocBench: A dataset for code-documentation alignment in software maintenance," in *Proc. Int. Conf. Mining Software Repositories (MSR), Data and Tool Showcase Track*, 2025.

[25] D. Baek, K. Krampf, and M. Pradel, "READU: Inconsistency-driven just-in-time detection and repair of README bugs," arXiv:2607.15780, 2026.

[26] "Tree-sitter documentation," https://tree-sitter.github.io/tree-sitter/, accessed: November 11, 2026.

[27] E. L. Kaplan and P. Meier, "Nonparametric estimation from incomplete observations," *J. American Statistical Association*, vol. 53, no. 282, pp. 457–481, 1958.

[28] D. R. Cox, "Regression models and life-tables," *J. Royal Statistical Society: Series B*, vol. 34, no. 2, pp. 187–202, 1972.

[29] P. K. Andersen and R. D. Gill, "Cox's regression model for counting processes: A large sample study," *The Annals of Statistics*, vol. 10, no. 4, pp. 1100–1120, 1982.

[30] C. Davidson-Pilon, "lifelines: Survival analysis in Python," *J. Open Source Software*, vol. 4, no. 40, p. 1317, 2019.

[31] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu *et al.*, "Qwen2.5-Coder technical report," arXiv:2409.12186, 2024.